

Le journal de l'ingénieur NexaCorp

30 jours qui ont changé sa façon de gérer le réseau

Ambient IT — Formation NetOps : Automatiser la gestion de son Infrastructure Réseau

Contents

Pourquoi ce journal	1
Semaine 1 — L'incident qui a tout déclenché	1
Semaine 2 — Faire parler tous les sites en même temps	2
Semaine 3 — Rendre chaque déploiement à l'épreuve des pannes	3
Semaine 4 — Passer à une vraie architecture, et aux outils Cisco	3
Semaine 5 — Fermer la boucle : datacenter et WAN	4
Ce que 30 jours ont changé	5
En résumé	5

Pourquoi ce journal

NexaCorp n'existe pas, mais son ingénieur réseau vous ressemblera bientôt. Plutôt que de vous présenter 20 compétences dans un tableau, on vous raconte 30 jours de sa vie : un problème concret, une solution qu'il découvre, un nouveau problème qui apparaît une fois le premier réglé. C'est exactement l'ordre dans lequel vous allez apprendre ces mêmes compétences cette semaine — chaque semaine du journal correspond à une journée de formation.

Gardez une question en tête en le lisant : **“est-ce que je saurai faire ça, moi, à la fin de cette semaine ?”**

Semaine 1 — L'incident qui a tout déclenché

(correspond au Jour 1 de la formation)

Jour 1. NexaCorp compte un seul ingénieur réseau pour trois sites : le siège, deux agences reliées en SD-WAN, et un datacenter. Ce trimestre, un changement de config-

uration doit être appliqué sur les deux agences. Pas d'outil d'automatisation, pas de lab de test — l'ingénieur prend la route, deux jours de déplacement, configure chaque site à la main.

Jour 3. De retour au siège, les utilisateurs de la première agence remontent des dysfonctionnements. Une partie de la configuration ne fonctionne pas comme prévu. Aucun moyen de savoir pourquoi à distance : pas de log centralisé, pas de script de diagnostic, rien. L'ingénieur reprend la route en urgence, passe la nuit sur place pour corriger et vérifier, avant de rentrer.

Jour 4. Cette fois, il se dit “plus jamais ça”. Premier chantier : arrêter de perdre la trace de ce qui change. Il met en place un **dépôt Git partagé**, avec un modèle de branches clair (feature → dev → staging → main). Chaque changement passe désormais par une pull request, relue avant d'être fusionnée. *Problème résolu : on sait enfin qui a changé quoi, et quand.*

Jour 5. Mais une pull request bien relue à l'œil n'empêche pas une faute de syntaxe de passer. Il ajoute un **pipeline CI/CD** à 4 étapes (lint, validate, test, deploy), avec le déploiement final bloqué derrière une approbation manuelle. *Problème résolu : plus aucun changement n'atteint la production sans passer par des vérifications automatiques.*

Jour 6. Le pipeline valide des fichiers, mais rien ne parle encore vraiment aux équipements. Il écrit un premier **client Python** capable d'interroger un routeur via son API REST — propre, avec gestion d'erreurs et logs, réutilisable. Comme il n'a pas d'équipement de test sous la main, il construit un **lab Docker** qui simule ses 3 sites, pour ne plus jamais tester directement en production. *Nouveau problème : chaque site se pilote encore un par un, à la main.*

Semaine 2 — Faire parler tous les sites en même temps

(correspond au Jour 2 de la formation)

Jour 7. Trois scripts à lancer, un par site, à chaque vérification : ça ne passera jamais à l'échelle si NexaCorp ouvre un quatrième site. L'ingénieur installe **Ansible**, avec un inventaire généré automatiquement et un rôle réutilisable. Une seule commande agit désormais sur les 3 sites à la fois. *Problème résolu : plus besoin de répéter la même opération trois fois.*

Jour 8. En préparant le prochain changement, il manque de saisir un VLAN invalide dans son fichier de variables — une erreur du même genre que celle qui avait causé l'incident du Jour 3. Il définit un **modèle YANG** qui décrit précisément ce qu'une configuration valide doit contenir, et un script qui rejette toute donnée incorrecte avant qu'elle n'atteigne Ansible. *Problème résolu : une erreur de saisie est bloquée à la source, pas découverte par un utilisateur des jours plus tard.*

Jour 9. Il se retrouve avec un fichier de configuration différent à maintenir pour chaque site — dès qu'un standard change, il faut modifier les trois. Il construit des **templates Jinja2** qui génèrent la configuration finale de chaque site à partir d'une seule source

de données. *Problème résolu : une seule source de vérité, plus de copier-coller entre sites.*

Jour 10. Un stagiaire qui reprend le dépôt tombe sur le mot de passe des routeurs, écrit en clair dans un fichier de variables. Il chiffre tous les identifiants avec **Ansible Vault** — désormais committables dans Git sans risque. *Nouveau problème : si un site tombe en panne au milieu d'un déploiement, rien ne garantit que les deux autres continuent d'être traités correctement.*

Semaine 3 — Rendre chaque déploiement à l'épreuve des pannes

(correspond au Jour 3 de la formation)

Jour 13. Justement : lors d'un test, l'agence 2 est injoignable, et le script plante entièrement — impossible de savoir si le siège et l'agence 1 se sont bien déployés. Il ajoute une gestion d'erreurs robuste (block/rescue/retries) : une panne sur un site est désormais interceptée et journalisée, sans jamais interrompre la vérification des autres. *Problème résolu : un site en panne n'empêche plus de traiter les deux autres.*

Jour 14. Mais il repense à l'incident du Jour 3 : rien n'empêche encore de pousser un changement défectueux sur les trois sites *en même temps*. Il met en place un déploiement **canary puis rolling** : le changement est d'abord testé sur un site pilote, et seulement s'il passe les vérifications, appliqué aux autres. En cas d'échec, la configuration précédente est restaurée automatiquement. *Problème résolu : exactement le scénario qui avait causé deux déplacements et une nuit blanche — désormais impossible.*

Jour 16. Les vérifications qu'il écrit à chaque déploiement commencent à se répéter d'un playbook à l'autre, sans réelle organisation. Il construit une vraie **suite de tests automatisés** — des pre-checks avant, des post-checks après, avec un rapport clair par site. *Problème résolu : tout ce qui a été construit se vérifie désormais en une seule commande.*

Jour 17. Un vrai incident survient malgré tout sur l'agence 1. Sans outil de diagnostic, il aurait fallu s'y déplacer. Il écrit un **script de diagnostic** qui interroge tous les sites, mesure la latence, et distingue clairement "site injoignable" d'"authentification refusée" d'"interface en panne". *Nouveau problème : tout ce travail repose sur un réseau de test à plat, qui ne reflète pas la vraie segmentation d'un réseau d'entreprise.*

Semaine 4 — Passer à une vraie architecture, et aux outils Cisco

(correspond au Jour 4 de la formation)

Jour 19. Le lab de test mélangeait tous les sites sur un seul réseau — pas très représentatif de la réalité, où chaque site a son propre LAN isolé, relié par un backbone. Il reconstruit son lab avec une vraie segmentation réseau. *Problème résolu : les tests reflètent enfin l'architecture réelle de NexaCorp.*

Jour 20. Un service critique tombe un soir, et personne n'est là pour le relancer avant le lendemain matin. Il migre ce service vers **Kubernetes** : plusieurs répliques, un load balancer, des sondes de santé, et une auto-réparation native — un pod qui tombe est recréé automatiquement, sans intervention humaine. *Problème résolu : un service critique reste disponible même en pleine nuit.*

Jour 21. La direction investit enfin dans un vrai contrôleur **Cisco DNA Center** pour gérer le LAN à grande échelle. L'ingénieur apprend son cycle d'authentification et d'action asynchrone, et étend son client Python pour piloter DNA Center comme il pilotait ses sites avec Ansible. *Problème résolu : les changements LAN passent par l'outil que la direction a choisi, avec la même rigueur que le reste.*

Jour 23. Il repense à l'incident du VLAN invité mal configuré, tout au début. Avec DNA Center, il automatise l'**onboarding SD-Access** : chaque nouvel hôte est rattaché automatiquement au bon réseau virtuel, sans configurer un VLAN à la main sur chaque switch. *Nouveau problème : tout ça couvre le LAN, mais le datacenter n'a toujours aucun outillage automatisé.*

Semaine 5 — Fermer la boucle : datacenter et WAN

(correspond au Jour 5 de la formation)

Jour 25. Un audit de sécurité pointe une règle oubliée dans la politique du datacenter (ACI) : deux applications qui ne devraient pas communiquer le peuvent, un contrat de sécurité manquant. L'ingénieur automatise le déploiement complet d'un tenant ACI (VRF, bridge domain, EPG, contrats) en un seul script, avec une vérification qui aurait détecté ce contrat manquant avant que l'audit ne le fasse. *Problème résolu : la sécurité du datacenter se déploie et se vérifie par script, plus par une checklist manuelle.*

Jour 27. En creusant les performances, il découvre plusieurs configurations avec des problèmes classiques : vitesse/duplex non forcés, spanning-tree portfast absent sur des ports d'accès. Il construit un **analyseur de configuration** qui détecte ces problèmes et corrige directement le template source. *Problème résolu : il ne corrige plus site par site, il corrige la source qui génère tout.*

Jour 28. Reste le sujet qui l'a mené jusqu'ici : le SD-WAN entre le siège et les agences. Il automatise la formation de la **topologie SD-WAN** (identifiants de site, transport, overlay) via l'API du contrôleur vManage, avec une vérification que chaque site rejoint bien le réseau. *Problème résolu : la topologie WAN se configure et se vérifie par script.*

Jour 30. Dernière pièce : une politique de routage SD-WAN mal réglée avait déjà saturé un lien par le passé. Il déploie une **policy centralisée** qui priorise le trafic métier sur tous les sites — avec une validation qui refuse d'activer une policy sur un site qui n'est pas encore dans l'overlay. *Le cercle est bouclé : LAN, datacenter et*

WAN sont désormais pilotés par automatisation, testés avant chaque déploiement, et corrigibles à distance, sans jamais reprendre la route.

Ce que 30 jours ont changé

Avant	Après
Déplacement de 2 jours pour un changement	Déploiement à distance et simultané (Semaine 2)
Configuration appliquée directement en prod	Validation automatisée avant déploiement (Semaines 2-3)
Problème découvert par les utilisateurs, des jours après	Post-checks automatisés en quelques secondes (Semaine 3)
Aucun retour en arrière possible	Rollback automatique dès l'échec d'un test (Semaine 3)
Déplacement d'urgence + nuit sur place	Diagnostic à distance en quelques secondes (Semaine 3)
Aucune trace des changements	Historique Git complet (Semaine 1)
Tout repose sur une seule personne	Scripts et pipelines documentés, partageables (Semaines 1-2)
VLAN invité mal configuré	Onboarding SD-Access automatisé (Semaine 4)
Contrat de sécurité datacenter oublié	Déploiement ACI avec vérification automatique (Semaine 5)
Politique SD-WAN mal réglée	Policy centralisée testée avant activation (Semaine 5)

En résumé

Vous n'allez pas passer une semaine à enchaîner 20 exercices sans lien entre eux. Vous allez vivre, en accéléré, ces mêmes 30 jours — chaque journée de formation correspond à une semaine du journal. Si un TP vous semble déconnecté du reste, revenez à la question de ce journal : **quel problème l'ingénieur de NexaCorp vient-il de rencontrer, et qu'est-ce qu'il apprend pour le résoudre ?**